

一、阅读文献概况

本次作业主要阅读了 SGLang、RadixAttention、vLLM PagedAttention 相关资料。通过学习，我了解到大语言模型推理并不只是简单的输入，输出，还涉及 Tokenizer、请求调度、Prefill、Decode、KV Cache 和显存管理等环节。

二、SGLang 与 RadixAttention

1. SGLang

SGLang 不只是用来运行大模型，它还对在线推理过程进行了优化。一个请求进入后，会经过 Tokenizer、Scheduler、Prefill 和 Decode 等过程。在这个过程中，KV Cache 会保存已经计算过的中间结果，后续可以继续使用。

2. RadixAttention 和 RadixCache

不同请求之间可能存在相同的前缀。如果每次都重新计算，就会产生重复工作。SGLang 的 RadixAttention / RadixCache 可以查找之前已经计算过的相同前缀，并复用对应的 KV Cache，减少相同内容带来的重复计算。

三、vLLM PagedAttention

1. PagedAttention 主要解决的问题

不同请求的长度不同，如果给每个请求提前分配一大片连续显存，就容易造成显存浪费。PagedAttention 借鉴操作系统分页的思路，把 KV Cache 分成较小的块，根据实际需求进行分配，从而提高显存利用率。

2. PagedAttention 和 RadixAttention 的区别

PagedAttention 解决的是 KV Cache 的显存管理问题。RadixAttention / RadixCache 主要利用相同前缀，减少重复计算。但两者最终都是为了提高大语言模型在线推理的效率。

四、Mooncake FAST' 25 Trace

Mooncake 的资料让我进一步了解了实际的大模型服务负载。Trace 中的 `input_length` 和 `output_length` 可以反映不同请求的输入、输出 Token 数量。

这次实验中，我根据部分 `trace` 数据构造测试请求，并使用 `Poisson` 到达过程模拟请求逐渐到达服务端。

五、结合本次实验后的理解

通过这次学习，我现在会把大语言模型在线推理理解成：

用户请求 → Tokenizer → Scheduler → Prefill → KV Cache → Decode → Sampling → 输出

当请求数量增加后，还需要考虑请求调度、KV Cache 管理、前缀复用以及 GPU 显存利用率等问题。

六、阅读后的个人体会

这次阅读让我第一次比较系统地认识了大语言模型在线推理的内部过程。结合这次实际搭建 `SGLang`、运行 `Qwen3-0.6B` 和测试 `Mooncake trace` 后，就能把理论和实际联系起来。我也认识到，阅读技术资料不能只记概念，最好结合代码、实验结果和自己遇到的问题一起理解，这样才能真正掌握一个技术。